

PEMANFAATAN ALGORITMA GREEDY DALAM PEMBUATAN BOT PERMAINAN ROBOCODE TANK ROYALE



Disusun Oleh:
Raymond Kolose Nathanael Simbolon
124140186
Teknik Informatika

PROGRAM STUDI TEKNIK INFORMATIKA
INSTITUT TEKNOLOGI SUMATERA

2026

DAFTAR ISI

BAB I.....	3
PENDAHULUAN.....	3
1.1 Deskripsi Tugas.....	3
BAB II.....	6
LANDASAN TEORI.....	6
2.1 Algoritma Greedy.....	6
2.2 Cara Kerja Program.....	6
BAB III.....	8
APLIKASI STRATEGI GREEDY.....	8
3.1 Mapping Persoalan Robocode Tank Royale ke Elemen Greedy.....	8
3.2 Eksplorasi Bot Solusi Greedy.....	9
3.3 Analisis Efisiensi dan Efektivitas Alternatif Solusi.....	10
BAB IV.....	12
IMPLEMENTASI DAN PENGUJIAN.....	12
4.1 Pseudocode Alternatif Solusi.....	12
4.2 Struktur Data, Fungsi, dan Prosedur Bot Utama.....	14
4.3 Pengujian Bot Utama.....	14
4.4 Analisis Hasil Pengujian.....	15
BAB V.....	17
KESIMPULAN.....	17
5.1 Kesimpulan.....	17
5.2 Saran.....	17

BAB I

PENDAHULUAN

1.1 Deskripsi Tugas

Pada Tugas ini, fokus utama ditekankan pada implementasi Algoritma Greedy (Algoritma Rakus). Di dalam arena pertempuran yang dinamis dan berubah dengan cepat, pengambilan keputusan yang instan berbasis optimum lokal (local optimum) menjadi sangat krusial. Melalui pendekatan Algoritma Greedy, bot dituntut untuk selalu mengambil keputusan terbaik dalam pertempuran dengan tujuan akhir memaksimalkan fungsi objektif, yaitu perolehan total skor (terdiri dari komponen Survival, Bullet Damage, dan Kill Bonus).

Untuk memberikan batasan yang jelas mengenai ruang lingkup pengerjaan, target capaian, serta aturan kompetisi yang harus dipenuhi dalam pengembangan bot ini, berikut adalah spesifikasi formal dari tugas yang diberikan:

a. Spesifikasi Wajib

- Buatlah 4 bot (1 utama dan 3 alternatif, tetap dikumpulkan 1 bot yang terbaik saja) dalam bahasa C# (.net) yang mengimplementasikan algoritma Greedy pada bot permainan Robocode Tank Royale dengan tujuan memenangkan permainan.
- Tugas dikerjakan berkelompok dengan anggota minimal 2 orang dan maksimal 3 orang.
- Strategi greedy yang diimplementasikan setiap kelompok harus dikaitkan dengan fungsi objektif dari permainan ini, yaitu memperoleh skor setinggi mungkin pada akhir pertempuran. Hal ini dapat dilakukan dengan mengoptimalkan komponen skor yang telah dijelaskan diatas.
- Strategi greedy yang diimplementasikan harus berbeda untuk setiap bot yang diimplementasikan dan setiap strategi greedy harus menggunakan heuristic yang berbeda.

- Bot yang dibuat TIDAK BOLEH sama dengan SAMPEL yang diberikan sebagai CONTOH. Baik dari starter pack maupun dari repository engine asli.
- Buatlah strategi greedy terbaik, karena setiap bot utama dari masing-masing kelompok akan diadu dalam kompetisi Tubes.
- Strategi greedy yang kelompok anda buat harus dijelaskan dan ditulis secara eksplisit pada laporan, karena akan diperiksa saat demo apakah strategi yang dituliskan sesuai dengan yang diimplementasikan.
- Setiap kelompok dapat menggunakan kreativitas yang bermacam macam dalam menyusun strategi greedy untuk memenangkan permainan. Implementasi pemain harus dapat dijalankan pada game engine yang telah disebutkan diatas serta dapat dikompetisikan dengan bot dari kelompok lain.
- Program harus mengandung komentar yang jelas, dan untuk setiap strategi greedy yang disebutkan, harus dilengkapi dengan kode sumber yang dibuat. Artinya semua strategi harus diimplementasikan.
- Mahasiswa disarankan membaca dokumentasi dari game engine. Perlu diperhatikan bahwa game engine yang digunakan untuk tubes ini SUDAH DIMODIFIKASI. Untuk Perubahan dapat dilihat pada bagian starter pack.

b. Spesifikasi Bonus (Opsional)

- (maks 10) Membuat video tentang aplikasi greedy pada bot serta simulasinya pada game kemudian mengunggahnya di Youtube. Video dibuat harus memiliki audio dan menampilkan wajah dari setiap anggota kelompok. Untuk contoh video tubes stima tahun-tahun sebelumnya dapat dilihat di Youtube dengan kata kunci “Tubes Stima”, “strategi algoritma”, “Tugas besar stima”,

dll. Semakin menarik video, maka semakin banyak poin yang diberikan.

- (maks 10) Beberapa kelompok pemenang lomba kompetisi akan mendapatkan nilai tambahan berdasarkan posisi yang diraih.
- Jika terdapat kesulitan selama mengerjakan tugas besar sehingga memerlukan bimbingan, maka dapat melakukan asistensi tugas besar kepada asisten (opsional). Dengan catatan asistensi hanya bersifat membimbing, bukan memberikan “jawaban”.
- Bot yang telah dibuat akan dikompetisikan dengan kelompok lain dan disaksikan oleh seluruh peserta kuliah. Terdapat hadiah menarik bagi kelompok yang memenangkan kompetisi.
- Program disimpan dalam repository yang bernama Tubes_NamaKelompok
- Sangat disarankan untuk menggunakan semantic commit. Buatlah release dengan format v1.x dengan x adalah nomor revisi dimulai dari revisi 0. Contoh v1.0 untuk release pertama, v1.1 untuk revisi selanjutnya.

BAB II

LANDASAN TEORI

2.1 Algoritma Greedy

Algoritma Greedy merupakan metode pemecahan masalah yang berprinsip pada pendekatan penyelesaian langkah demi langkah (step-by-step). Pada setiap langkahnya, algoritma ini akan mengambil keputusan yang paling menguntungkan saat itu juga tanpa memikirkan konsekuensi jangka panjang di masa depan. Konsep ini dikenal dengan istilah *Local Optimum* (optimum lokal), dengan harapan bahwa akumulasi dari setiap pilihan lokal tersebut akan mengarah pada *Global Optimum* (optimum global) atau solusi terbaik di akhir penyerahan masalah.

2.2 Cara Kerja Program

Aplikasi yang dikembangkan berjalan di atas game engine Robocode Tank Royale berbasis .NET C#. Game ini memiliki arsitektur *event-driven* (berbasis kejadian), di mana bot akan terus mengulang siklus pencarian menggunakan radar.

[Siklus Run Loop] -> [Radar Scan] -> [OnScannedBot Event] -> [Eksekusi Greedy & Tembak]

Secara umum, bot bekerja melalui tahapan berikut:

- Inisialisasi Independen: Memisahkan komponen bodi, meriam, dan radar (`AdjustRadarForGunTurn = true`, dll.) agar bot tidak mengalami stuck/blocking saat bergerak secara bersamaan.
- Sensing (Radar Scan): Pada fungsi `Run()`, radar diputar 360° secara kontinu menggunakan fungsi non-blocking `SetTurnRadarRight()`.
- Processing & Decision Making: Ketika radar mendeteksi bot musuh, event `OnScannedBot` akan dipicu. Di sinilah Algoritma Greedy bekerja menggunakan kriteria heuristik tertentu untuk mengunci target.

- Acting (Predictive & Execution): Setelah target ditentukan oleh fungsi seleksi Greedy, bot menghitung posisi masa depan musuh dengan rumus trigonometri linear, memutar meriam secara simultan, dan mengeksekusi tembakan via SetFire().

BAB III

APLIKASI STRATEGI GREEDY

3.1 Mapping Persoalan Robocode Tank Royale ke Elemen Greedy

Untuk mempermudah pemodelan matematis, komponen permainan Robocode dipetakan ke dalam elemen-elemen formal algoritma Greedy sebagai berikut:

- Himpunan Kandidat (C): Seluruh robot musuh yang terdeteksi oleh radar dalam arena pada suatu tick/frame pertempuran.

$$C = \{ BOT_1, BOT_2, \dots, BOT_n \}$$

- Himpunan Solusi (S): Maksimal 1 robot musuh terpilih yang memenuhi kriteria heuristik untuk dijadikan target utama serangan pada tick tersebut.
- Fungsi Seleksi: Fungsi yang mengevaluasi seluruh kandidat berdasarkan kriteria heuristik tertentu (misalnya, mencari nilai minimum jarak).
- Fungsi Kelayakan: Memeriksa apakah target musuh masih aktif ($Energy > 0$) dan posisinya berada di dalam koordinat batas arena pertempuran.
- Fungsi Solusi: Memastikan apakah eliminasi atau kerusakan yang diberikan pada target berhasil meningkatkan perolehan poin bot kita.
- Fungsi Objektif: Memaksimalkan total skor akhir pertempuran ($Total_Score$) yang dirumuskan sebagai:

$$Total_Score = Survival_Score + Damage_Score + Kill_Bonus$$

3.2 Eksplorasi Bot Solusi Greedy

Berikut adalah tabel dari keempat bot yang digunakan beserta kriteria heuristik dan logika Greedy yang diimplementasikan.

Nama Bot	Kriteria Heuristik	Logika Greedy
Runtuh (Utama)	Jarak Terdekat ($\min(Distance)$)	Mengincar musuh terdekat karena probabilitas peluru meleset paling kecil, serta memaksimalkan kerusakan (Damage Score).
Killa	Energi Terendah ($\min(Energy)$)	Mengincar musuh dengan darah paling sedikit demi mengamankan Kill Bonus secepat mungkin.
Sniper	Kecepatan Terlambat ($\min(Abs(Speed))$)	Mengincar musuh yang sedang diam atau bergerak lambat demi menghemat energi peluru dan menaikkan tingkat akurasi (Bullet Hit Rate).
Koboi	Sudut Putar Terkecil ($\min(Abs(\Delta\theta_{gun}))$)	Mengincar musuh yang posisinya paling sejajar dengan moncong meriam saat ini demi meminimalkan delay putaran senjata untuk tembakan instan

3.3 Analisis Efisiensi dan Efektivitas Alternatif Solusi

Berikut merupakan analisis efisiensi serta efektivitas alternatif solusi Greedy dari keempat bot yang digunakan:

- Efisiensi Komputasi: Keempat alternatif solusi memiliki efisiensi komputasi yang sangat tinggi ($O(1)$ untuk setiap event scan) karena hanya melakukan perbandingan nilai primitif (double) saat robot musuh terpantul di radar.
- Efektivitas Skor:
 - Heuristik Jarak (Runtuh) sangat efektif mengoptimalkan *Damage Score* karena deviasi peluru di jarak dekat mendekati nol.
 - Heuristik Energi (Killa) efektif mengoptimalkan *Kill Bonus*, namun kurang efektif jika musuh yang sekarat berada sangat jauh di ujung peta.
 - Heuristik Kecepatan (Sniper) menghemat konsumsi energi, namun rentan kalah jika musuh yang lambat ternyata memiliki armor/firepower yang besar.
 - Heuristik Sudut (Koboi) unggul dalam hal kecepatan tembak instan, namun abai terhadap posisi bahaya diri sendiri.

3.4 Strategi Bot Utama

Pada tugas ini, bot Runtuh dengan heuristik jarak terdekat dipilih sebagai bot utama daripada 3 bot alternatif lainnya. Alasan dipilihnya Runtuh sebagai bot utama adalah karena berdasarkan karakteristik penilaian *Robocode*, komponen *Damage Score* berkontribusi linear terhadap penambahan energi diri kita sendiri (melalui mekanisme bullet hit energy return). Dengan memilih musuh terdekat, peluru prediksi berpeluang sangat tinggi untuk mengenai sasaran. Hal ini secara otomatis menjaga stabilitas energi bot utama sekaligus memberikan tekanan kerusakan yang

konsisten bagi lawan, memenuhi fungsi objektif untuk meraup nilai Survival dan Damage tertinggi.

BAB IV

IMPLEMENTASI DAN PENGUJIAN

4.1 Pseudocode Alternatif Solusi

Berikut adalah kumpulan pseudocode dari keempat bot yang digunakan dalam tugas ini:

- Bot Runtuh (Heuristik Jarak)

```
INITIALIZE closestDistance = INFINITY
INITIALIZE targetId = NULL

ON ScannedBot(EnemyBot):
    distance = CALC_DISTANCE(Bot.X, Bot.Y, EnemyBot.X,
    EnemyBot.Y)

    // Jika belum ada target ATAU musuh ini lebih dekat
    IF targetId == NULL OR EnemyBot.Id == targetId OR distance
    < closestDistance THEN
        targetId = EnemyBot.Id
        closestDistance = distance

        TURN_GUN_TOWARDS(EnemyBot)
        FIRE_BULLET()
    END IF
```

- Bot Killa (Heuristik Energi/Darah)

```
INITIALIZE lowestEnergy = INFINITY
INITIALIZE targetId = NULL

ON ScannedBot(EnemyBot):
    // Jika belum ada target ATAU musuh ini memiliki darah
    lebih tipis
    IF targetId == NULL OR EnemyBot.Energy < lowestEnergy THEN
        targetId = EnemyBot.Id
        lowestEnergy = EnemyBot.Energy

        TURN_GUN_TOWARDS(EnemyBot)
        FIRE_BULLET()
    END IF
```

- Bot Sniper (Heuristik Kecepatan Terlambat)

```

INITIALIZE slowestSpeed = INFINITY
INITIALIZE targetId = NULL

ON ScannedBot(EnemyBot):
    // Jika belum ada target ATAU musuh bergerak lebih lambat
    IF targetId == NULL OR EnemyBot.Speed < slowestSpeed THEN
        targetId = EnemyBot.Id
        slowestSpeed = EnemyBot.Speed

        TURN_GUN_TOWARDS(EnemyBot)
        FIRE_BULLET()
    END IF

```

- Bot Koboi (Heuristik Sudut *Aiming*)

```

INITIALIZE minAngle = INFINITY
INITIALIZE targetId = NULL

ON ScannedBot(EnemyBot):
    angleDifference = ABS(CALC_BEARING_TO(EnemyBot) -
        Bot.GunDirection)

    // Mencari musuh yang paling searah dengan moncong meriam
    saat ini
    IF targetId == NULL OR angleDifference < minAngle THEN
        targetId = EnemyBot.Id
        minAngle = angleDifference

        TURN_GUN_TOWARDS(EnemyBot)
        FIRE_BULLET()
    END IF

```

4.2 Struktur Data, Fungsi, dan Prosedur Bot Utama

Bot Runtuh mengimplementasikan Alternatif 1 dengan spesifikasi arsitektur sebagai berikut:

- Struktur Data Status (State Variables): Menggunakan dua properti global primitif yaitu `int? targetId` untuk menyimpan ID musuh optimal (solusi) dan `double closestDistance` sebagai pembanding jarak minimum sementara.
- Prosedur `Run()`: Merupakan siklus hidup utama robot yang berisi infinite loop. Di dalamnya terdapat generator angka acak (`Random()`) yang menjalankan prosedur pergerakan blocking (Maju, Mundur, Belok Kiri, Belok Kanan) secara fluktuatif untuk mengecoh kalkulasi musuh (Erratic Movement).
- Fungsi `OnScannedBot(ScannedBotEvent)`: Merupakan event handler yang mengeksekusi logika fungsi seleksi Greedy. Fungsi ini akan menghitung jarak Euclidean menggunakan parameter input koordinat X dan Y dari objek event. Apabila jaraknya valid (lebih kecil dari `closestDistance`), maka prosedur `TurnGunRight` dan `Fire` akan diinterupsi/dieksekusi menuju target.
- Prosedur `OnHitWall()` dan `OnBotDeath()`: Digunakan untuk me-reset state jarak dan target id sebagai bentuk penyegaran (pengaturan ulang dari himpunan kandidat kosong) terhadap lingkungan yang berubah.

4.3 Pengujian Bot Utama

Pengujian dieksekusi sebanyak minimal 3 kali ronde pertempuran dengan format 1 vs 1 melawan bot asisten di arena. Hasilnya:

- Pengujian 1 (Pergerakan Jarak Jauh): Bot musuh berada di sisi ujung arena yang berseberangan dengan Runtuh.

Kejadian Unik: Pada fase ini, tembakan bot Runtuh sering meleset karena jarak yang terlalu jauh (Waktu tempuh peluru lambat),

meskipun laras meriam sudah membidik secara optimal. Namun Runtuh berhasil bermanuver menghindari tembakan beruntun musuh berkat gerak acaknya.

- Pengujian 2 (Konfrontasi Jarak Dekat/Melee): Bot Runtuh secara acak bergerak mendekati musuh hingga jarak di bawah 200 piksel.

Kejadian Unik: Algoritma Greedy langsung mengunci target dan menembakkan firepower maksimal (Power 3). Karena target berada di jarak dekat, peluru menghantam musuh telak meskipun musuh mencoba maju-mundur secara tidak beraturan.

- Pengujian 3 (Kejadian Terdesak/Hit Wall): Bot Runtuh tidak sengaja bergerak menuju sudut dinding akibat pergerakan random.

Kejadian Unik: Kejadian unik tertangkap saat bot menyentuh dinding, fungsi OnHitWall langsung membatalkan gerak maju dan melakukan sentakan mundur secara paksa menjauhi dinding. Dalam sepersekian detik, radar menemukan musuh yang mengejar dari belakang, dan bot langsung melakukan serangan balasan secara optimal.

4.4 Analisis Hasil Pengujian

Berdasarkan serangkaian pengujian di atas, penerapan Strategi *Greedy by Closest Distance* secara umum berhasil mendapatkan nilai optimal (kemenangan damage yang signifikan) apabila dihadapkan pada skenario pertarungan jarak dekat hingga menengah. Di jarak ini, kelemahan sistem bidik tanpa prediksi masa depan dapat ditutupi oleh kecepatan luncur roket yang teramat cepat (sehingga musuh tidak memiliki jeda menghindar).

Berdasarkan hasil observasi di arena pertempuran, kelemahan mendasar dari sifat myopic (rabun dekat) yang menjadi ciri khas algoritma Greedy terlihat jelas ketika objek musuh berada pada jarak yang ekstrem atau sangat jauh di ujung arena. Dalam skenario spasial tersebut, meskipun probabilitas akurasi proyektil sangat rendah karena adanya jeda waktu tempuh peluru, bot Runtuh tetap mengeksekusi perintah tembakan secara konstan akibat pembacaan status yang mengidentifikasi objek tersebut sebagai target "terdekat" pada saat itu. Ketidakmampuan algoritma dalam memprediksi perpindahan posisi musuh di jarak jauh ini mengakibatkan proyektil selalu meleset dari sasaran, yang pada akhirnya memicu penurunan energi bot secara drastis (*energy drop*) akibat pemborosan sumber daya amunisi untuk situasi taktis yang tidak menguntungkan.

BAB V

KESIMPULAN

5.1 Kesimpulan

Setelah melalui seluruh tahapan perancangan, implementasi kode program, hingga proses pengujian performa melalui simulasi battle berskala multi-bot di arena Robocode Tank Royale, dapat ditarik beberapa poin kesimpulan akademis mengenai efisiensi dan efektivitas bot Runtuh sebagai berikut:

- Pemetaan permasalahan bot Robocode ke dalam elemen algoritma Greedy telah berhasil dilakukan, di mana optimasi pemilihan target tembakan difokuskan pada kedekatan jarak koordinat Euclidean musuh.
- Dari eksplorasi empat alternatif solusi, didapatkan bahwa Greedy by Closest Distance adalah metode yang paling relevan, efisien, dan memiliki hit-rate tertinggi jika dipadukan dengan strategi Erratic Random Movement.
- Implementasi algoritma Greedy ini sangat optimal pada pertarungan jarak dekat, namun memperlihatkan kelemahan pemborosan firepower pada penargetan musuh yang berada di jarak ekstrem (jauh dari jangkauan efisien peluru).

5.2 Saran

Bagi pengembangan bot selanjutnya, direkomendasikan untuk tidak menggunakan algoritma Greedy secara tunggal murni. Sistem penargetan dapat disempurnakan dengan menggabungkan algoritma pencarian heuristik adaptif contohnya *Linear Predictive Targeting* atau *Pattern Matching*, di mana bot hanya akan menggunakan tembakan langsung (Greedy-Direct) pada jarak dekat, dan beralih ke tembakan prediktif pada jarak jauh untuk meminimalisir peluru yang terbuang percuma.

DAFTAR PUSTAKA

- Larsen, F. N. (2022). Robocode Tank Royale API. Diakses melalui:
<https://robocode-dev.github.io/tank-royale/>
- Munir, R. (2015). Strategi Algoritmik. Penerbit Informatika. Bandung.
- Nelson, M. (2001). Robocode: Game Programming in Java. IBM alphaWorks.

LAMPIRAN

Tautan Repository GitHub: <https://github.com/colossiansr/reruntuh.git>